CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Chandubhai S. Patel Institute of Technology (CSPIT)

Department of Computer Science & Engineering

ITUE203 – Web Development Frameworks

PRACTICAL 10

PHP Login System with Role-Based Access Control, Session Management and Secure Logout

PHP 8 sessions · password_verify() · MySQLi · permission-based RBAC (admin / editor / student) · CSRF · audit log

Corrected submission – implemented in the integrated StudentHub portal (studenthub/)

Student Name	Shreyas
Roll Number	25CS023
Student ID	25CS023
Email	25cs023@charusat.edu.in
Course	ITUE203 – Web Development Frameworks
Programme	B.Tech Computer Science & Engineering
Semester / Academic Year	3 / 2026-27
Date of Submission	
Faculty Signature	

Corrected report. This report replaces the original Practical 10 report (`Practical-10/PRAC_10_ITUE203_25CS023.docx`, kept unchanged in the repository). The original report describes a standalone login in `Practical-10/` with its own database `studenthub_prac10`, an admin and a student dashboard, `timeout.php` and the cookie `P10SESSID`. This corrected version describes the same requirements as the one authentication system of the integrated portal (`studenthub/includes/auth.php`, database `studenthub_db`): sign-in by username or e-mail, the cookie `STUDENTHUBSESSID`, permission-based access for admin, editor and student, an inactivity timeout checked on every request, logout by POST only, and the later fixes that block deactivated students and disabled accounts. All screenshots in this report were taken from the canonical integrated application `studenthub/` with Microsoft Edge on disposable test databases with fictional records; all test results are copied from the saved test outputs of the repository.

1. Aim and Objectives

To build the StudentHub login system in PHP and MySQL with password verification, role-based access control, secure session management with an inactivity timeout and a secure logout – as the single sign-in of the integrated portal that protects every page.

- Authenticate users by username or e-mail and password with MySQLi prepared statements and `password_verify()`, with one generic message for every failed attempt.
- Regenerate the session ID immediately after a successful login (protection against session fixation).
- Send each role to its own start page and refuse pages of a higher role on the server with HTTP 403 "Access Denied".
- End inactive sessions automatically and destroy the session completely on logout; no protected page may be shown again after logout, not even with the Back button.
- Integration: re-check the account on every request, protect every POST form with a CSRF token and record security events in the audit log.

2. Problem Definition

After registration (Practical 9) users must be able to sign in. A careless login reveals which accounts exist, keeps the same session ID before and after login (an attacker who planted an ID takes over the session), decides access only by hiding links, leaves sessions open forever on shared computers and lets the browser show a cached page after logout.

In the integrated portal two more problems existed (TEST_REPORT §5): all practicals shared one PHP session on `localhost:8000`, so signing in to one application could authorise another that only checked the role; and admin actions used GET links without CSRF tokens. The portal also has three roles (admin, editor, student) instead of two, and an account's role or status can change while its owner is signed in.

3. Tools and Technologies

Tool	Use in this practical
PHP 8.2.12 (XAMPP)	Sessions, `password_verify()`, permission checks; PHP's built-in server for the tests
MySQLi (exception mode), MariaDB 10.4.32	Prepared statements on `users`, `students` and `audit_logs` in `studenthub_db`
HTML5, CSS3	Sign-in page, 403 page and logout confirmation (`assets/css/auth.css`); no JavaScript is needed
Python 3 (standard library)	`tests/test_auth.py` – HTTP tests with separate cookie jars, four PHP servers
Microsoft Edge (headless) +	`tests/browser_auth.py`, `tests/browser_inactive_login.py`,

Playwright	`tests/browser_e2e.py` – browser checks and screenshots
Git / GitHub	Commit `8011d39` (Practicals 9 and 10 integrated); later `6420efc` (inactive students) and `05927b3` (account status, Practical 14)

4. Place in the Integrated Portal

File	Role in Practical 10
`includes/auth.php`	The one authentication and authorisation system: secure session, CSRF, login/logout helpers, per-request re-validation, timeout, permissions, password policy
`login.php`	Sign-in form and controller; fixed status messages
`logout.php`	Logout by POST with the CSRF token; GET shows a confirmation
`403.php`	The shared "Access Denied" page (HTTP 403)
Every protected page	`require_permission('portal.view')` – dashboard, profile, courses, attendance, assignments, results, achievements, notifications, directory, Event Finder, Data Explorer, poster delivery
`admin_dashboard.php`	`require_permission('admin.console')`, plus a permission per action and `require_valid_csrf()` for every POST
Public pages	`index.php`, `login.php`, `register.php`, `contact.php` use `optional_user()` (never redirect)
`audit_logger.php`	`log_audit()` for sign-in, logout, timeout and access events

Only `includes/auth.php` calls `session_start()` – a static test checks that no page starts its own session – so every page shares one session with the same rules.

5. Accounts and Data Used

Sign-in uses the `users` table of `studenthub_db`: `id`, `username` and `email` (both UNIQUE – either can be used to sign in), `password` (the bcrypt hash written by registration or `tools/create_demo_user.php`), `role` (admin, editor or student) and `last_login`, which is updated after each successful sign-in. Two later additions are read too: `users.status` (active/disabled, migration 004, Practical 14) and the status of the student record linked through `students.user_id` (Practical 11). The session stores only `user_id`, `username`, `email`, `role`, `last_activity` and `csrf_token`. Security events are written to `audit_logs`:

Audit action	Status	When
USER_LOGIN / USER_LOGOUT	SUCCESS	successful sign-in / logout
LOGIN_FAILED	FAILED	wrong password or unknown account (no user_id is stored)
ACCESS_DENIED	SECURITY_BLOCK	a signed-in role lacks the permission of the requested page or action
CSRF_REJECTED	SECURITY_BLOCK	a POST without a valid token (forms and logout)
SESSION_TIMEOUT	INFO	the session was idle longer than the timeout
LOGIN_DENIED / SESSION_REVOKED	SECURITY_BLOCK	later fixes: correct password but the account is disabled or the linked student record is inactive

6. Login and Password Verification

`login.php` accepts only a POST with the session's CSRF token and both fields filled. A value containing `@` is looked up as an e-mail address, anything else as a username – always with a prepared statement. For an unknown account a dummy bcrypt hash is checked, so both failures take about the same time.

`password_verify()` reads algorithm, cost and salt from the stored hash and compares; the hash cannot be turned back into the password.

studenthub/login.php (lines 29–69) – lookup, password_verify(), session start, generic message

```
29          // An e-mail address contains "@", a username never does
30          $column = str_contains($entered, "@") ? "email" : "username";
31          $stmt = $conn->prepare("SELECT u.id, u.username, u.email, u.password, u.role, u.status AS
    account_status, s.status AS student_status
32                                  FROM users u LEFT JOIN students s ON s.user_id = u.id WHERE u.$column
    = ? LIMIT 1");
33          $stmt->bind_param("s", $entered);
34          $stmt->execute();
35          $account = $stmt->get_result()->fetch_assoc();
36          $stmt->close();
37
38          // An unknown account is checked against a dummy hash, so both failures take about the same
    time
39          $hash = $account["password"] ??
    '$2y$10$cL5pLAsT9eF311SYAHI/9OCtAW62X11zkDZ3VA2wDn4DbvPI3.YlC';
40          $verified = password_verify($password, $hash);
41          // Correct password, but the account is disabled or its linked student record is deactivated:
    no session is started
42          // (generic message; the status is only revealed to someone who knows the password)
43          $block  = $account !== null && $verified ? account_block_reason($account) : null;
44          $denied = $block !== null;
45          if ($denied) {
46              log_audit($conn, (int) $account["id"], $account["username"], "LOGIN_DENIED", "Sign-in
    refused: $block.", "SECURITY_BLOCK");
47          } elseif ($account !== null && $verified && in_array($account["role"], STUDENTHUB_ROLES,
    true)) {
48              login_user($account);
49              $up = $conn->prepare("UPDATE users SET last_login = NOW() WHERE id = ?");
50              $up->bind_param("i", $account["id"]);
51              $up->execute();
52              $up->close();
53              log_audit($conn, (int) $account["id"], $account["username"], "USER_LOGIN", "User logged
    in with role=" . $account["role"], "SUCCESS");
54              if ($account["role"] === "admin") {
55                  flash("success", "Welcome, " . $account["username"] . ". You are signed in as
    administrator.");
56                  redirect("admin_dashboard.php");
57              }
58              flash("success", "Welcome back, " . $account["username"] . ".");
59              redirect("dashboard.php");
60          }
61          if ($denied) {
62              $error = LOGIN_STATUS["access_disabled"][1];
63          } else {
64              $error = "Invalid username/email or password.";
65              log_audit($conn, null, mb_substr($entered, 0, 100), "LOGIN_FAILED",
66                  $account === null ? "Sign-in attempt for an unknown account." : "Wrong password.",
    "FAILED");
67          }
68      }
69      http_response_code($denied ? 403 : 401);
```

Situation	Answer	Message / destination
Correct credentials, role admin	303	`admin_dashboard.php` – "Welcome, … You are signed in as administrator."
Correct credentials, editor or student	303	`dashboard.php` – "Welcome back, …."
Wrong password, unknown account or e-mail, SQL-injection text	401	"Invalid username/email or password." (LOGIN_FAILED)
Missing / wrong CSRF token; an empty field	401	"The sign-in form expired. Please try again." / "Please enter both …"
Correct password, account disabled or student record inactive (later)	403	"Your access to StudentHub is currently disabled. Please contact the administrator."

Messages shown after a redirect are chosen by a short code from the fixed list `LOGIN_STATUS` (`login_required`, `timeout`, `logged_out`, `account_unavailable`, `access_disabled`); text from the address is never printed. `redirect()` accepts only a relative `page.php?…` path of the portal, so `login.php?next=https://evil.example/` cannot redirect elsewhere (tested).

7. Session Management

`start_secure_session()` configures the cookie before the session starts: the name `STUDENTHUBSESSID` (the portal no longer shares the session of the practical folders), HttpOnly (not readable by JavaScript), SameSite=Strict (not sent with requests from other sites), Secure when HTTPS is used, and strict mode, so IDs not created by the server are refused and IDs in the URL are never accepted. Signed-in pages send headers that forbid framing, MIME sniffing and caching:

studenthub/includes/auth.php (lines 40–77) – timeout setting, cookie flags and security headers

```
40  // Inactivity timeout in seconds (default 30 minutes; STUDENTHUB_SESSION_TIMEOUT overrides it, e.g.
    for tests)
41  function session_timeout_seconds(): int
42  {
43      $value = (int) (getenv('STUDENTHUB_SESSION_TIMEOUT') ?: 1800);
44      return max(1, $value);
45  }
46
47  function start_secure_session(): void
48  {
49      if (session_status() === PHP_SESSION_ACTIVE) {
50          return;
51      }
52      ini_set('session.use_strict_mode', '1');      // IDs not created by this server are refused
53      ini_set('session.use_only_cookies', '1');     // never accept a session ID from the URL
54      ini_set('session.use_trans_sid', '0');
55      session_name('STUDENTHUBSESSID');
56      session_set_cookie_params([
57          'lifetime' => 0,
58          'path'     => '/',
59          'secure'   => !empty($_SERVER['HTTPS']) && $_SERVER['HTTPS'] !== 'off',
60          'httponly' => true,
61          'samesite' => 'Strict',
62      ]);
63      session_start();
64  }
65
66  // Security headers. Private pages are also kept out of the browser cache, so the Back button cannot
    show them after logout.
67  function send_security_headers(bool $private): void
68  {
69      if (headers_sent()) {
70          return;
71      }
72      header('X-Frame-Options: DENY');
73      header('X-Content-Type-Options: nosniff');
74      header('Referrer-Policy: same-origin');
75      if ($private) {
76          header('Cache-Control: no-store, no-cache, must-revalidate, max-age=0, private');
77          header('Pragma: no-cache');
```

After the password is verified, `login_user()` empties the session, calls `session_regenerate_id(true)` – the old ID is deleted on the server, so an ID known before login is useless – and stores the account data read from the database (never from the form) with a new CSRF token. `end_session()` is the logout counterpart:

studenthub/includes/auth.php (lines 153–182) – reset_session(), end_session() and login_user()

```
153 // Empties the session and gives it a new ID (used when a session is no longer valid)
154 function reset_session(): void
155 {
156     $_SESSION = [];
157     session_regenerate_id(true);
158 }
```

```
159
160 // Logout: clears the data, expires the cookie in the browser and deletes the session on the server
161 function end_session(): void
162 {
163     $_SESSION = [];
164     $params = session_get_cookie_params();
165     setcookie(session_name(), '', [
166         'expires' => time() - 42000, 'path' => $params['path'], 'domain' => $params['domain'],
167         'secure' => $params['secure'], 'httponly' => $params['httponly'], 'samesite' =>
    $params['samesite'],
168     ]);
169     session_destroy();
170 }
171
172 // Successful login or registration: new session ID (session fixation), new CSRF token, details from
    the database
173 function login_user(array $account): void
174 {
175     $_SESSION = [];
176     session_regenerate_id(true);
177     $_SESSION['user_id']       = (int) $account['id'];
178     $_SESSION['username']      = (string) $account['username'];
179     $_SESSION['email']         = (string) $account['email'];
180     $_SESSION['role']          = (string) $account['role'];
181     $_SESSION['last_activity'] = time();
182     $_SESSION['csrf_token']    = bin2hex(random_bytes(32));
```

Every protected request runs `validate_session()`. It ends an idle session, then **re-reads the account** with its linked student record: a deleted account loses access at once, a role change applies at the next request, and a disabled account or deactivated student is signed out. Only then is the activity time renewed.

studenthub/includes/auth.php (lines 227–250) – per-request timeout and account re-validation

```
227 // Returns [user or null, reason]: reason is '' (valid or no session), 'timeout',
    'account_unavailable' or 'access_disabled'
228 function validate_session(): array
229 {
230     $user = current_user();
231     if ($user === null) {
232         return [null, ''];
233     }
234     if (session_has_expired()) {
235         global $conn;
236         log_audit($conn, $user['id'], $user['username'], 'SESSION_TIMEOUT', 'Session ended after ' .
    session_timeout_seconds() . ' s of inactivity.', 'INFO');
237         reset_session();
238         return [null, 'timeout'];
239     }
240     [$fresh, $reason, $detail] = refresh_user_from_database($user);
241     if ($fresh === null) {
242         if ($reason === 'access_disabled') {
243             global $conn;
244             log_audit($conn, $user['id'], $user['username'], 'SESSION_REVOKED', "Session ended:
    $detail.", 'SECURITY_BLOCK');
245         }
246         reset_session();                                  // session data deleted on the server, new
    empty session ID
247         return [null, $reason];
248     }
249     $_SESSION['last_activity'] = time();
250     return [$fresh, ''];
```

8. Role-Based Access Control

Access is decided on the server by explicit permissions, not by hidden links. The list as it is now (the `inquiries.manage` entry was added with the Practical 7 integration):

studenthub/includes/auth.php (lines 19–37) – permissions per role

```
19 // Every permission and the roles that hold it. An editor has only what is listed here (no admin
    rights).
20 const STUDENTHUB_PERMISSIONS = [
```

```
 21      'portal.view'     => ['admin', 'editor', 'student'],   // dashboard, profile, academic and campus
     pages
 22      'admin.console'   => ['admin'],                        // Admin Console (admin_dashboard.php)
 23      'students.manage' => ['admin'],
 24      'events.manage'   => ['admin'],
 25      'users.manage'    => ['admin'],
 26      'audit.view'      => ['admin'],
 27      'inquiries.manage' => ['admin'],                       // stored Help & Contact inquiries
     (Practical 7)
 28  ];
 29
 30  function e($value): string
 31  {
 32      return htmlspecialchars((string) $value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
 33  }
 34
 35  function role_can(?string $role, string $permission): bool
 36  {
 37      return $role !== null && in_array($role, STUDENTHUB_PERMISSIONS[$permission] ?? [], true);
```

Every protected page starts with `require_permission()`. A visitor without a valid session is redirected to the sign-in page (`status=login_required`, `timeout`, `account_unavailable` or `access_disabled`); a signed-in user whose role lacks the permission receives the shared 403 page and an ACCESS_DENIED entry. A role field added to a form changes nothing, because the role always comes from the database.

studenthub/includes/auth.php (lines 261–280) – require_login() and require_permission()

```
261  function require_login(): array
262  {
263      start_secure_session();
264      send_security_headers(true);
265      [$user, $reason] = validate_session();
266      if ($user === null) {
267          redirect('login.php?status=' . ($reason !== '' ? $reason : 'login_required'));
268      }
269      return $user;
270  }
271
272  function require_permission(string $permission): array
273  {
274      $user = require_login();
275      if (!role_can($user['role'], $permission)) {
276          global $conn;
277          log_audit($conn, $user['id'], $user['username'], 'ACCESS_DENIED',
278              "Role '{$user['role']}' lacks permission '$permission' for " .
     basename($_SERVER['SCRIPT_NAME'] ?? ''), 'SECURITY_BLOCK');
279          include dirname(__DIR__) . '/403.php';
280          exit;
```

Role	Start page after sign-in	Allowed	Refused (HTTP 403 + ACCESS_DENIED)
admin	`admin_dashboard.php` (Admin Console)	all portal pages and the Admin Console	–
editor	`dashboard.php`	the student portal pages (`portal.view` only)	Admin Console and all its POST actions
student	`dashboard.php`	the student portal pages	Admin Console and all its POST actions

Later, Practical 14 added role-based navigation (`includes/navigation.php`): menu rows from `menu_items` are shown up to the user's rank (student 1 < editor 2 < admin 3) and links to missing pages are hidden – the pages behind the links still check the permission on the server.

9. CSRF Protection

Each session has one random token (32 bytes from `random_bytes()`, 64 hexadecimal characters), placed in a hidden field of every POST form by `csrf_field()`. `hash_equals()` compares in constant time. POST handlers call `require_valid_csrf()`: a missing or wrong token stops the request with HTTP 403 "Security check failed", nothing is changed and CSRF_REJECTED is logged (Figure 8). `login.php`, `register.php` and `logout.php` use `csrf_is_valid()` directly and answer with their own message or redirect.

studenthub/includes/auth.php (lines 82–111) – token, comparison and the POST guard

```
 82  function csrf_token(): string
 83  {
 84      if (!isset($_SESSION['csrf_token']) || !is_string($_SESSION['csrf_token'])) {
 85          $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
 86      }
 87      return $_SESSION['csrf_token'];
 88  }
 89
 90  function csrf_field(): string
 91  {
 92      return '<input type="hidden" name="csrf_token" value="' . e(csrf_token()) . '">';
 93  }
 94
 95  function csrf_is_valid($submitted): bool
 96  {
 97      $expected = $_SESSION['csrf_token'] ?? null;
 98      return is_string($expected) && is_string($submitted) && $submitted !== '' &&
     hash_equals($expected, $submitted);
 99  }
100
101  // For POST handlers: a missing or wrong token stops the request (HTTP 403) and is written to the
     audit log
102  function require_valid_csrf(): void
103  {
104      if (csrf_is_valid($_POST['csrf_token'] ?? null)) {
105          return;
106      }
107      global $conn;
108      log_audit($conn, isset($_SESSION['user_id']) ? (int) $_SESSION['user_id'] : null,
     $_SESSION['username'] ?? 'guest',
109          'CSRF_REJECTED', 'POST to ' . basename($_SERVER['SCRIPT_NAME'] ?? '') . ' without a valid
     token', 'SECURITY_BLOCK');
110      studenthub_fail(403, 'Security check failed', 'This form has expired or was not sent from
     StudentHub. Go back, reload the page and try again. Nothing was changed.');
111      exit;
```

10. Session Timeout and Secure Logout

The inactivity timeout is 30 minutes (1800 s); the environment variable `STUDENTHUB_SESSION_TIMEOUT` overrides it, which the tests use with 2 seconds. A request after the limit logs SESSION_TIMEOUT, empties the session, gives it a new ID and leads to the sign-in page with "Your session ended because you were inactive for too long. Please sign in again."

Logout accepts only a POST with the session's token, so a link or image on another site cannot sign the user out. The sidebar's Logout button and "Sign out & switch account" on the 403 page are such forms (`logout_button()`); opening `logout.php` directly shows a confirmation button instead. `end_session()` empties `$_SESSION`, sends an expired cookie with the same flags and destroys the session on the server, so a kept copy of the old cookie no longer works. Because signed-in pages are sent with `Cache-Control: no-store`, the Back button after logout leads to the sign-in page.

studenthub/logout.php (lines 11–21) – logout only by POST with a valid token

```
 11  if ($_SERVER["REQUEST_METHOD"] === "POST") {
 12      if (!$user) {
 13          redirect("login.php");
 14      }
 15      if (!csrf_is_valid($_POST["csrf_token"] ?? null)) {
```

```
16          log_audit($conn, $user["id"], $user["username"], "CSRF_REJECTED", "Logout request without a
    valid token (ignored).", "SECURITY_BLOCK");
17          redirect($user["role"] === "admin" ? "admin_dashboard.php" : "dashboard.php");
18      }
19      log_audit($conn, $user["id"], $user["username"], "USER_LOGOUT", "User logged out successfully.",
    "SUCCESS");
20      end_session();
21      redirect("login.php?status=logged_out");
```

11. Later Extensions: Deactivated Students and Disabled Accounts

Two later steps extended the same system. Commit `6420efc` (TEST_REPORT §8) blocks a `student` account whose linked record was deactivated in the Admin Console (Practical 11); commit `05927b3` (Practical 14) lets administrators disable any account (`users.status`). Both use `account_block_reason()`. At sign-in the block is checked only **after** `password_verify()` succeeded: the answer is HTTP 403 with one generic message, no session is started and LOGIN_DENIED is logged, while a wrong password still gets the normal message – so the status is revealed only to someone who knows the password. An open session ends at its next request (session data deleted, new empty ID, SESSION_REVOKED). Admin and editor accounts linked to an inactive record and student accounts without a record are not affected by the student-record rule (Figures 6–7). After reactivation, or after an administrator enables the account again, the student signs in normally (tested in `browser_inactive_login.py` and `test_dashboard.py`).

12. Output Screenshots

Genuine full-page captures from Microsoft Edge (headless, Playwright, window 1280 × 900) of the integrated portal on PHP's built-in server with disposable databases built from `database/studenthub_demo.sql`; all accounts are fictional and password fields are masked or empty. Sources: Figures 1, 3 and 5 `tests/browser_auth.py`; Figures 2, 4 and 8 `tests/browser_e2e.py`; Figures 6 and 7 `tests/browser_inactive_login.py`. There is no screenshot of the logout message or of the timeout; both are verified by the HTTP and browser checks in Section 13.

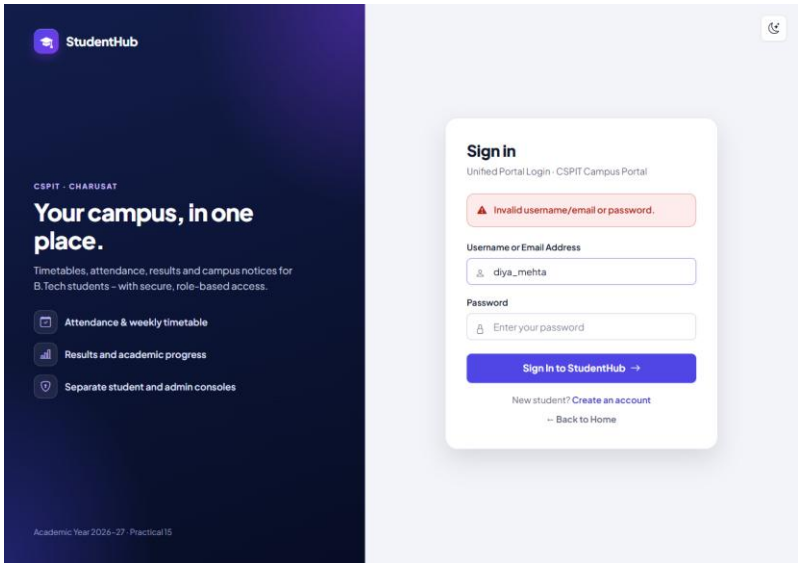


Figure 1: Wrong password for diya_mehta: the generic message "Invalid username/email or password." (HTTP 401) – the same for an unknown account; the username stays in the field, the password field is empty (login-invalid.png)

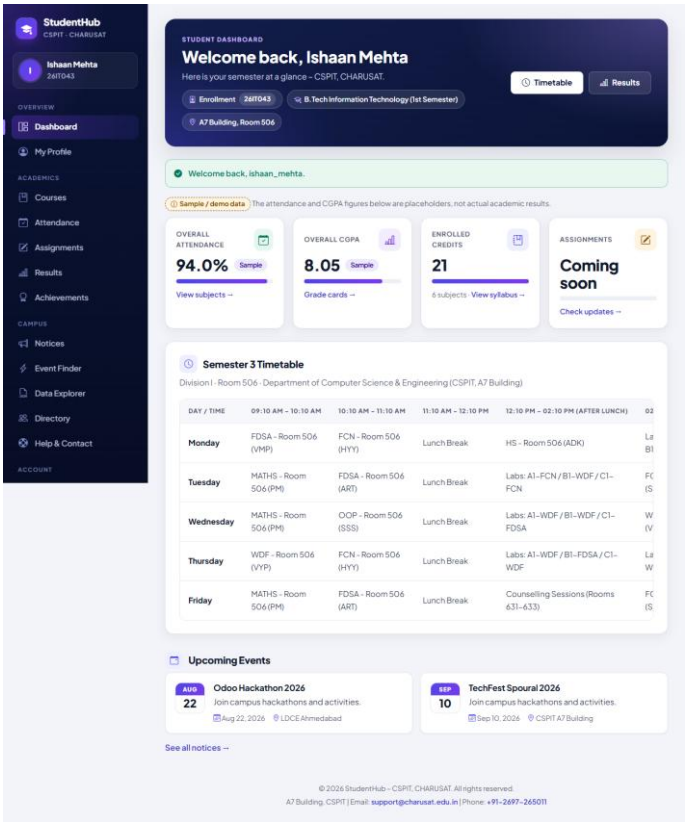


Figure 2: Sign-in with the e-mail address ishaan.mehta@example.com after a POST logout: 303 to dashboard.php with the one-time message "Welcome back, ishaan_mehta."; the sidebar shows the linked record (Ishaan Mehta, 26IT043) (02-login-dashboard.png)

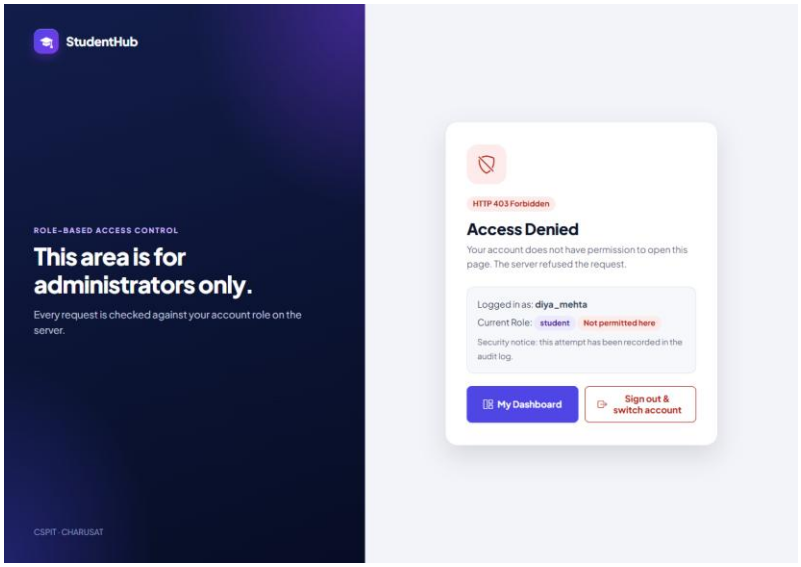


Figure 3: Student diya_mehta opening admin_dashboard.php directly: HTTP 403 "Access Denied" (403.php), role student "Not permitted here", the attempt recorded in the audit log; "Sign out & switch account" is a POST logout form (student-admin-denied.png)

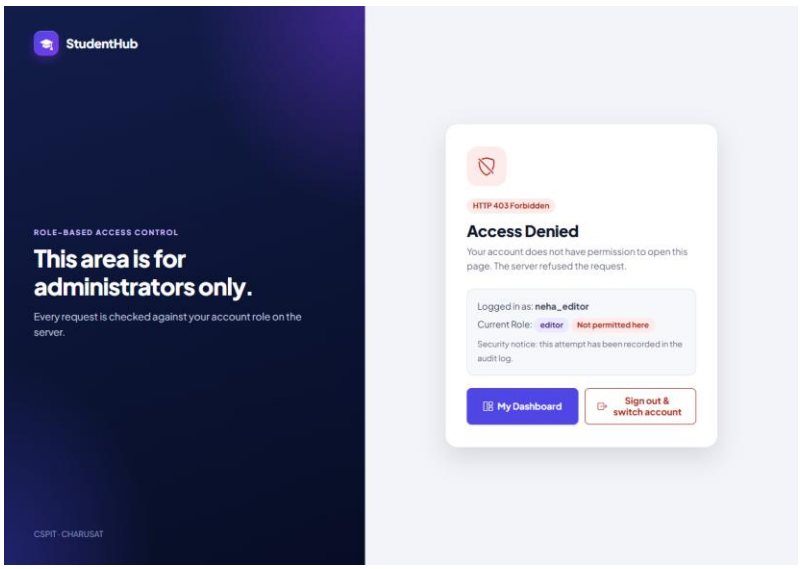


Figure 4: Editor neha_editor opening the Admin Console: the same HTTP 403 page – an editor holds only portal.view (04-editor-access-denied.png)

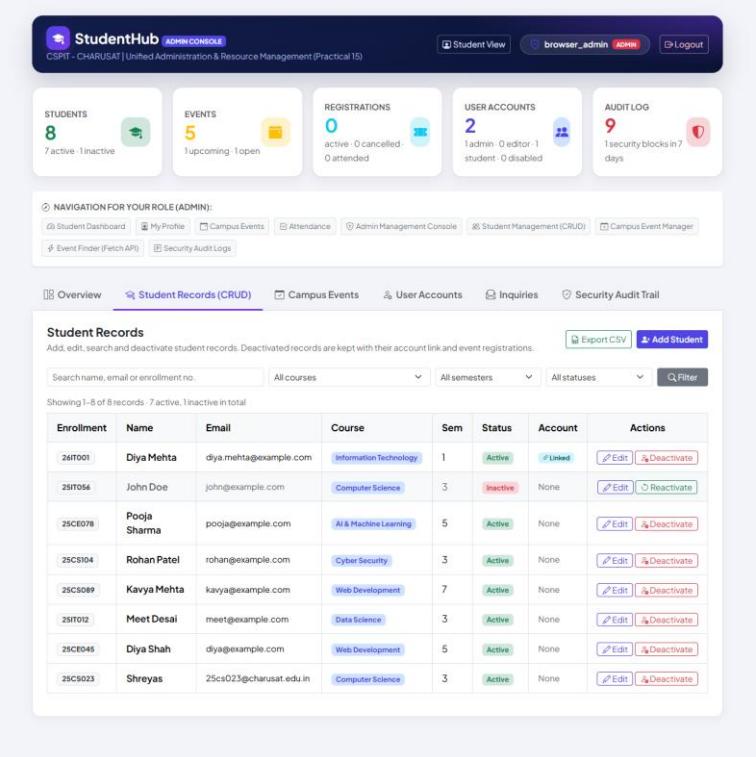


Figure 5: Administrator browser_admin after sign-in: the Admin Console, here on the Student Records tab, where every status change is a POST form with the CSRF token (checked by the test); later integration steps added further tabs (admin-console.png)

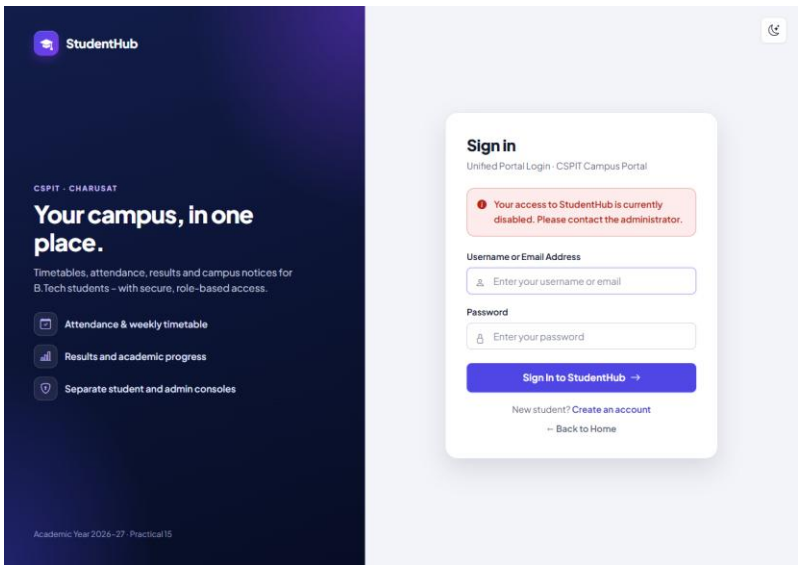


Figure 6: Later fix: after the administrator deactivated Lina Mehra's linked record, her open session's next request ends on the sign-in page with "Your access to StudentHub is currently disabled. Please contact the administrator." (15-session-ended-after-deactivation.png)

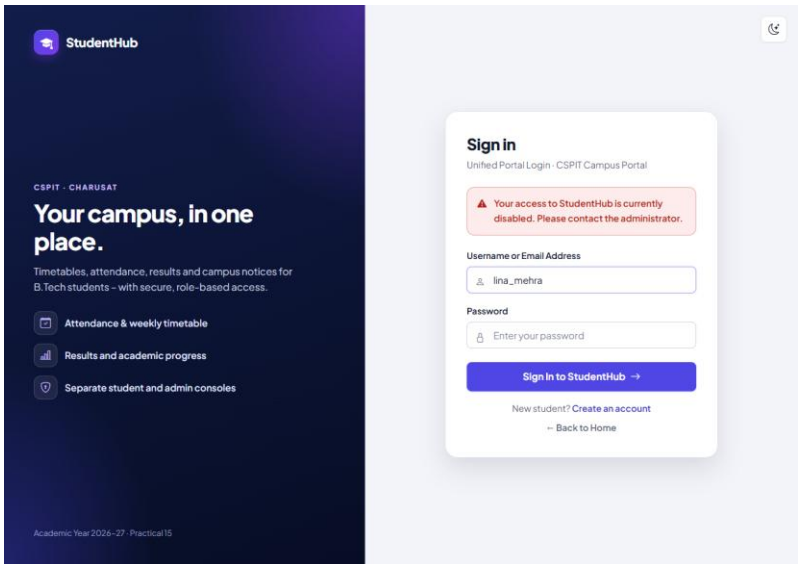


Figure 7: Signing in again as lina_mehra with the correct password while the record is Inactive: refused with the same generic message (HTTP 403), no session started (16-inactive-login-denied.png)

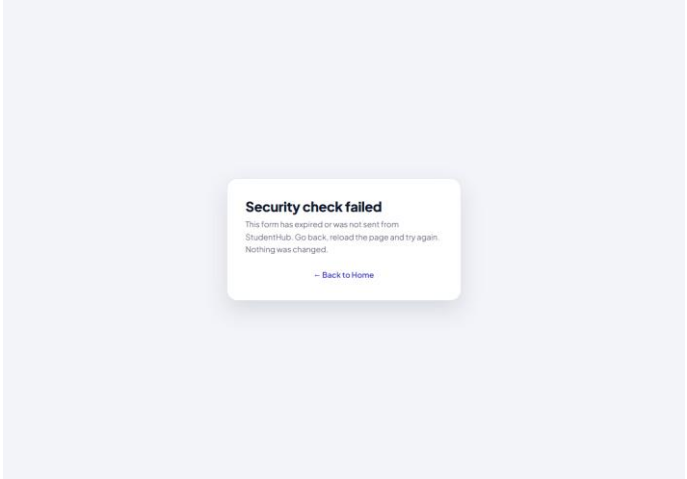


Figure 8: A forged form without the CSRF token, submitted to the Admin Console from the administrator's signed-in browser: "Security check failed" (HTTP 403, require_valid_csrf()); the target student stayed Active (17-csrf-rejected.png)

13. Testing and Results

All tests were executed on 31 October 2026 with PHP 8.2.12 (XAMPP), MariaDB 10.4.32 and PHP's built-in server on Windows 11, against disposable databases (`studenthub_auth_test` for `test_auth.py`, built from `database/studenthub_demo.sql` and dropped afterwards). `test_auth.py` runs four servers: a normal one, one with a 2-second timeout, and two whose database is unavailable (unknown database, wrong password). The live `studenthub_db` users and password hashes were unchanged.

Suite	Result	Output
`tests/test_auth.py` – registration, login, sessions, RBAC (HTTP)	**132 / 132 passed**	`tests/test_auth_output.txt`
`tests/browser_auth.py` – Microsoft Edge	**14 / 14 passed**	`tests/browser_auth_output.txt`
`tests/test_students.py` – incl. 22 sign-in checks of the later fix	**170 / 170 passed**	`tests/test_students_output.txt`
`tests/browser_inactive_login.py` – Edge: deactivate, session ended, refused, reactivate	**13 / 13 passed**	`tests/browser_inactive_login_output.txt`
`tests/run_all.py` – complete run of every suite	**841 passed, 0 failed**	`tests/run_all_output.txt`

Area of test_auth.py	Checks	Examples (from test_auth_output.txt)
Login	16	CSRF token; cookie STUDENTHUBSESSID HttpOnly + SameSite=Strict; wrong password, unknown account, unknown e-mail and SQL injection → 401 with the same message; LOGIN_FAILED; no token → refused; old ID ≠ new ID and the pre-login ID gives no access; `last_login` + USER_LOGIN; e-mail login, admin → Admin Console; no-store, DENY, nosniff; no open redirect or reflected text (3)
Logout	6	GET shows a confirmation; wrong token does not log out; token → "You have been signed out."; every protected page then redirects; replayed old cookie → no access; USER_LOGOUT
Role-based access control	33	11 protected pages without session → sign-in page; 4 public pages stay public; contact form POST without token → 403; all pages 200 for student, editor and admin with tokens in every POST form; student and editor → 403 + ACCESS_DENIED and their admin POSTs refused; admin console and 4 tabs
CSRF and admin operations	10	admin POST without token → 403 "Security check failed", logged; message via the session; escaping; a GET link cannot deactivate; POST without token refused; with token → soft delete; own account cannot be deleted
Session re-validation	3	promotion and demotion apply at the next request; an account deleted while signed in loses access at once
Profile / password change	6	own data only; wrong current or weak new password refused; no token → 403; new hash stored and session ID regenerated; old password no longer works
Inactivity timeout (2 s)	3	requests within the limit keep the session; after 3 s idle → sign-in page with the timeout message; session gone, SESSION_TIMEOUT logged
Database unavailable / clean-up	11	unknown database or wrong password → HTTP 503 without details and without a fallback; real reason only in the log, the password never; disposable database removed; live database untouched
Static checks, registration, server log	7 + 34 + 3	only auth.php starts sessions; no GET logout link; every protected page calls `require_permission()`; no message text from the URL; registration (Practical 9 report); no PHP warnings

Browser checks (`browser_auth.py`, 14): the sidebar Logout button (POST) signs out; the Back button after logout does not show the dashboard; wrong password → generic message, correct login → dashboard; student → HTTP 403 "Access Denied"; admin → Admin Console; student status changes are POST forms with a CSRF token; admin logout; no JavaScript errors except the intended 401/403/422 answers (plus one known 404 for `/favicon.ico`). The complete run lists "Practicals 9/10 – registration, login, sessions, RBAC" 132 / 0, "Edge – registration, login, RBAC" 14 / 0 and "Edge – inactive student login" 13 / 0.

14. Problems Found and Fixed

- **Shared session:** all practicals and the portal shared one PHP session on `localhost:8000`; the portal now uses its own cookie `STUDENTHUBSESSID` (TEST_REPORT §5 → §6).
- **Account enumeration:** the login page revealed whether an account exists; every failure now gets the same message and HTTP 401, and unknown accounts are checked against a dummy hash (§5 → §6).
- **GET actions without CSRF:** admin delete actions used GET links without tokens; every state change is now a POST with the session token, and logout is POST-only (§5 → §6).
- **Untested timeout:** the inactivity timeout was not tested (§4); it is now tested with a 2-second server (§6).
- **Stale permissions:** the role was taken only at sign-in; the account is now re-read on every request, so promotion, demotion and deletion apply immediately (§6).
- **Deactivated students could still sign in** (§7) – fixed in commit `6420efc` (§8), extended to disabled accounts by Practical 14.

15. Differences from the Original Practical and Limitations

Aspect	Original report (Practical-10/)	Integrated portal
Database / login name	Own `studenthub_prac10`; e-mail only	Shared `studenthub_db`; username or e-mail
Roles and dashboards	admin / student; each refused the other's dashboard	admin / editor / student with permissions; admins may also open the student pages; students and editors are refused the Admin Console
Session	`P10SESSID`; regeneration shown as fingerprints on the dashboard	`STUDENTHUBSESSID`; regeneration verified by tests (no IDs shown)
Timeout	300 s; `timeout.php` and a meta refresh	1800 s (configurable), checked on every request; the session ends at the next request
Account changes	role read at login only	account, role and status re-read on every request
Tests	76 / 76	132 / 132 HTTP, 14 / 14 + 13 / 13 Edge; complete run 841 / 0

- There is no limit on repeated sign-in attempts (TEST_REPORT §6, §13); remember-me is not offered.
- The Secure cookie flag is set only over HTTPS; all tests used plain HTTP on PHP's built-in server (Apache deployment is described in `DEPLOYMENT.md` but was not part of these runs).
- Editors currently hold only the student-portal permission.

16. Conclusion

The portal has one authentication system for every page. Users sign in by username or e-mail with prepared statements and `password_verify()` and get one generic message for every failure; the session ID is regenerated at login and the cookie is HttpOnly and SameSite=Strict. Access is decided on the server by explicit permissions – students and editors receive HTTP 403 for the Admin Console and every refusal is

logged – and the account is re-read on every request, so role changes, deletions, disabled accounts and deactivated students take effect immediately. Idle sessions end after the timeout, logout is a CSRF-protected POST that destroys the session, and no-store headers keep the Back button from showing a protected page. 132 HTTP checks, 14 + 13 browser checks and the complete run of 841 checks passed.

17. References

- Project repository: https://github.com/TheEmperor08/StudentHub-ITUE203-25CS023 – `studenthub/includes/auth.php`, `studenthub/login.php`, `studenthub/logout.php`, `studenthub/403.php`, `tests/`, `evidence/`, `TEST_REPORT.md` §6 and §8; commits `8011d39`, `6420efc`, `05927b3`
- PHP Manual – password_verify: https://www.php.net/manual/en/function.password-verify.php
- PHP Manual – session_regenerate_id: https://www.php.net/manual/en/function.session-regenerate-id.php
- PHP Manual – Session security: https://www.php.net/manual/en/features.session.security.management.php
- OWASP – Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP – Authentication Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP – Cross-Site Request Forgery Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html